University of Geneva

Faculty of Science

Series 7: Genetic Programming

Metaheuristics for Optimization
14x013

Michel Jean Joseph Donnet

2025-12-31

Contents

1 Introduction

Technological advances in recent years have enabled humans to solve complex real-life problems through relevant problem modeling, allowing them to harness the power of computers. However, some concrete problems are too complex for a computer to find a solution in polynomial time.

For example, finding the analytical form of a function for which only certain output values are known requires checking different types of functions with different types of values, which necessitates searching an astronomical search space. This problem has practical applications in many fields, such as physics and economics.

This is why certain metaheuristic methods have been developed to deal with the complexity of this type of problem with a huge search space, providing a sufficiently good solution in polynomial time. One of the metaheuristic approaches, called genetic programming, involves imitating the evolution of populations in nature and applying it to the analytical form of a function in order to find an analytical form that corresponds to the result of the given function. The idea behind this concept is inspired by biology: let's assume we have an initial generation of individuals. The best individuals in the population are more likely to survive thanks to natural selection. This is called the selection stage. Next, the individuals reproduce, creating offspring that is a mixture of the parents' genetic material. Thanks to this mixture, the offspring may have better characteristics than their parents. This is called the crossover stage. In addition, the offspring have certain mutations that have altered their genetic material, making them better (or worse) than their parents. This stage is called mutation. Finally, the new generation will produce a new generation, and so on, which will improve certain characteristics that allow individuals to survive better. The genetic programming metaheuristic works in a similar way with generations: an initial generation of programs is created. Then, programs are selected with a higher chance of being selected for good programs during the selection stage. Next, programs are mixed in pairs during the crossover stage. Mutations are applied to the newly created programs, and the stages are repeated for the new generation, and so on.

This work will focus on a specific problem that deals only with Booleans and basic operations on them. A set of variables and operators is provided to construct the function, and the objective is to find a correct assignment of operators and variables that reproduces the given binary table. In our case, we only have 4 variables with 4 operators: "AND", "OR", "NOT", and "XOR". So the search space for the given problem corresponds to all possible combinations of these variables and operators, in accordance with the program length constraint: the combination must not exceed the maximum length specified.

To evaluate each program generated by the combination of operators and variables, the fitness function is defined. The fitness function is simply a comparison between the result generated by the program and the expected function: the fitness of a given program corresponds to the number of results that match the expected results. This means that the maximum fitness searched corresponds to the size of the dataset to be matched.

2 Methodology

The objective of the work is to find a logical expression composed of Boolean variables and operators that correspond to the given binary table. To search for the logical expression, we use the genetic programming metaheuristic. Since we are dealing with logical expressions, the generated logical expressions may not be valid, which is why the Section 2.1 addresses this issue.

The basic principle of genetic programming is to evolve an initial population of programs or logical expressions in order to find a logical expression whose binary table corresponds to the given binary table. The operations performed on the initial population to make it evolve are selection, mutation, and crossover.

2.1 Validity

First, how should invalid expressions generated by the genetic programming algorithm be handled ?

Several options can be used, such as regenerating the expression until it is valid, modifying the existing expression to make it valid, or even ignoring invalid expressions…

The method chosen in this work to address validity consists of evaluating the expression step by step, ignoring errors and faults such as operators that do not have a sufficient number of variables to be executed, or others.

In this way, even if the generated expression is not valid, it may be partially valid and composed of good patterns. Since the generated expression has aptitude due to partial evaluation, it can be selected to be mixed with other expressions to create a good valid expression close to the optimal expression.

For instance, if we are looking for the expression `["X1", "X2", "AND", "X3", "AND", "X4", "AND"]` which means $X1 \wedge X2 \wedge X3 \wedge X4$, the expression `["AND", "X2", "AND", "X3", "AND", "X4", "AND"]` is invalid because the first operator have no parameters. However, by ignoring the errors and evaluating the expression step by step, the method used will evaluate the program `["X2", "X3", "AND", "X4", "AND"]` which corresponds to $X2 \wedge X3 \wedge X4$ which is close to the expression searched for.

2.2 Initial Population

The choice of the initial population affects the quality of genetic programming, because if the initial population created is close to the desired expression, the algorithm will converge quickly, whereas if the initial population is far from the desired expression, the algorithm will take longer to converge.

Several approaches can be used, such as generating only valid expressions, generating at least one valid expression, or constraining a percentage of the population to be valid expressions.

The method chosen in this work to generate the initial population is to generate totally randomly the initial population with no restrictions on validity of generated expressions. The basic idea is to select randomly operators and variables and then shuffle them together to create a random expression. However, since an operator generally requires two variables, this means that the expression generally requires more variables than operators. This is why a threshold has been set to ensure that, for instance, 1/3 of the generated expression consists of operators and 2/3 of variables. In the work, the threshold was set at 1/3.

2.3 Selection

The selection step is crucial for the convergence of the genetic programming algorithm. If the selected expressions are very poor, the algorithm may not converge because the selection will move the algorithm away from the target expression.

The selection step allows the algorithm to exploit good expressions by selecting them. The fitness of an expression gives an idea of the quality of that expression, as it indicates the similarity of the expression to the target expression. Different types of expression selection can be performed, such as random selection of expressions or stochastic selection of expressions by ranking them according to their fitness.

The method used in this work consists of taking k expressions at random and selecting the best expression. This method is called k-tournament selection, and the parameter k can be used to control the balance between exploration and exploitation.

Indeed, if k is equal to the size of the population, this means that each time, only the best expression will be selected, which favors exploitation. On the contrary, if k is equal to 1, this means that each time, an expression will be selected at random, which favors exploration.

However, a form of elitism is also applied, because in the selection process used in this work, the best expression is automatically selected.

2.4 Crossover

The crossover step control determines how two expressions are mixed when generating offspring. This step is therefore important in the genetic programming algorithm, as it introduces a certain amount of diversity by mixing expressions, but also exploits expressions by performing crossovers that do not introduce changes in both expressions. This is why the crossover step allows for exploration and exploitation.

The method chosen for crossover is called single-point crossover. Suppose we have two parental expressions $P1$ and $P2$ of size l. Single-point crossover will randomly select a number k in [1, l]. Then, the first offspring will consist of the "genetic material" $[1, k]$ from $P1$ and $[k + 1, l]$ from $P2$, and the second offspring will consist of $[1, k]$ from $P2$ and $[k + 1, l]$ from $P1$, as illustrated on Figure 1.

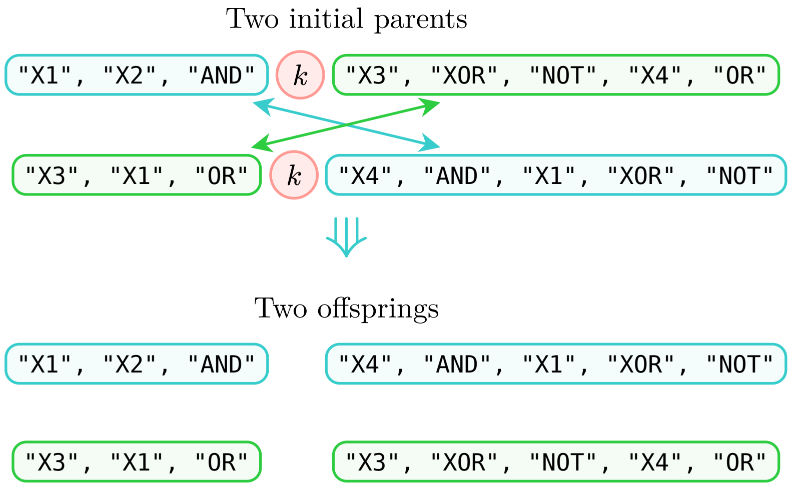

Figure 1: Single-point crossover

2.5 Mutation

The mutation stage introduces a certain amount of diversity into the population in order to promote exploration in the genetic programming algorithm. This stage prevents the algorithm from getting stuck in a suboptimal solution by maintaining a certain amount of diversity.

There are several approaches to performing mutation, such as replacing a randomly selected part of the expression with something newly generated.

The approach used involves taking each expression one by one, then selecting each element of the expression one by one. The selected element then has a chance of being mutated, determined by the guided parameter `p_m`, which gives the probability of performing a mutation.

The mutation itself has a 50% chance of replacing the selected element with an operator and a 50% chance of replacing it with a variable.

In this way, a level of diversity is always maintained by the mutation step.

3 Implementation

The genetic programming algorithm implemented follows the genetic algorithm Algorithm 1.

Algorithm 1: Genetic Programming

```
population = generate_initial_population(population_size = 80)
max_iterations = 100
current_iteration = 0
while current_iteration < max_iterations
    population = selection(population, tournament_size = k)
    population = crossover(population, p_c = 0.6)
    population = mutation(population, p_m = 0.1)
    current_iteration ++
end
return population, best_individu
```

In the given Algorithm 1, the `generate_initial_population` function follows Algorithm 2, the `selection` function follows Algorithm 3, the `crossover` function follows Algorithm 4, and the `mutation` function follows Algorithm 5.

Algorithm 2: Generation of initial population

```
population_size = 80
program_length = 15
threshold = 1/3
operators = [AND, OR, XOR, NOT]
variables = [X1, X2, X3, X4]
population = []
while length(population) < population_size
    select randomly program_length × threshold elements from operators
    select program_length × (1 - threshold) elements from variables
    add them to individual
    shuffle individual
    add individual to population
end
return population
```

Algorithm 3: Selection step

```
k = 2 (number of individuals selected for the tournament)
new_population = []
add best element of population to new_population
while length(new_population) ≠ length(population)
    randomly select k individuals from the population
    add the best of the selection to the new population
end
return population
```

Algorithm 4: Crossover step

```
p_c = 0.6 (probability to perform crossover)
i = 0
n = length(population)
while i < n
    if random_number() < p_c
        apply single-point crossover as explained in Section 2.4
        on current element i and the next one (i + 1) modulo n
    i + 2
end
return population
```

Algorithm 5: Mutation step

```
p_m = 0.1 (probability to perform a mutation)
for individual in population
    for element in individual
        if random_number() < p_m
            if random_number() < 0.5
                randomly select a variable and replace the element with it
            else
                randomly select an operator and replace the element with it
            end
        end
    end
end
return population
```

As shown in the pseudocodes, there are different types of parameters that can be used to adjust the genetic programming algorithm. To adjust the parameters correctly, an average of 1,000 trials was calculated for each experiment in order to get a good idea of the impact of the parameter and how to adjust it. The adjusted parameters are as follows:

- **Population size**. It is very important to define the population size correctly, as this has a considerable impact on computation time. Indeed, having a large population is not useful if it does not improve the results, as it requires more calculations. However, a population that is too small will make the algorithm less effective. That is why certain tests were carried out on populations of 10, 20, 40, 50, 80, 100, 120, 150, and 200 individuals in order to obtain an overview of the impact of population size on genetic programming.
- **Program length**. The program length must be correctly defined. If the program length is too small, the target expression will never be found if it requires an expression larger than the program length. Conversely, if the program length is too large, which is not a problem in our case thanks to the management of expression validity as described in Section 2.1, the computational effort will be greater than necessary and the effectiveness of the algorithm could be reduced. The parameters tested are the following program lengths: 5, 8, 10, 12, 15, 20, 25, and 30.
- **k-tournament selection**. During the selection step, selection by k-tournament is performed as described in Section 2.3. The parameter k, which determines the number of individuals chosen for the tournament, controls the balance between exploration and exploitation. Indeed, with k equal to the population size, the best individual will always be chosen, while a small k corresponds to a random selection. The tests performed are as follows: 2-tournament, 4-tournament, 6-tournament, 8-tournament, 10-tournament and 20-tournament selection.
- **Crossover probability `p_c`**. As used in Algorithm 4, `p_c` indicates the probability of performing a crossover on a group of two parents. The crossover applied is a single-point crossover, well illustrated by Figure 1. Tests on the crossover probability are performed for values of 0, meaning no crossover, but also for values of 0.2, 0.4, 0.6, 0.8, and 1.
- **Mutation probability `p_m`**. As used in Algorithm 5, `p_m` describes the probability of performing a mutation on an element of an individual in the population. Mutation helps maintain a level of diversity in the population, which is why this parameter must be defined carefully. In order to define this parameter correctly, several experiments were conducted with mutation probabilities of 0, i.e., no mutation, but also with probabilities of 0.05, 0.01, 0.1, and 0.3.

Some tests were also carried out on the combined length of the program and the size of the population in order to find the best parameters.

For fitness, all completely invalid expressions are simply ignored in order to avoid a situation where fitness does not increase due to certain individuals having a fitness of 0 because of invalid expressions.

4 Results

Best solution: [‘X2’, ‘X1’, ‘AND’, ‘X3’, ‘X2’, ‘XOR’, ‘OR’, ‘NOT’, ‘X4’, ‘AND’]

Length 10: Fitness=16.0, Solution=[‘X1’, ‘X2’, ‘AND’, ‘X2’, ‘X3’, ‘XOR’, ‘OR’, ‘NOT’, ‘X4’, ‘AND’]

5 Discussion

(2.0)

- Pm, Pc, Iterations
- Population Size, Tournament Size
- Program length, variable length (what must we change if program is not fixed size ??? theoretical… No need to implement it…)

6 Conclusion

(0.25)